Lab 1 – Password Cracking: Results

Cybersecurity and Defense-in-Depth

Student:	Ahmed Hossam Abdelrahman
Course:	Cybersecurity and Defense-in-Depth
TA:	Ziad Hazem
Environment:	Windows 10, Python 3.13
Submission Date:	25 October 2026

On Verification

Every cracked credential is checked against the provided login program. `Login.pyc` ships as Python **3.12** bytecode (magic `cb 0d 0d 0a`), which cannot be imported under the installed 3.13 interpreter, so verification is run through `LoginTemplate.py` – the exact source the `.pyc` was compiled from. Both read the same `../.loginCheck` oracle and apply the same scheme `H(user,pw) = sha256_iter(sha256(user||pw), 90000)`, so a `Login successful.` line proves the credential is valid on our system.

Q1 — Crack Magdy via `MostCommonPWs`

```
[Q1] user=Magdy  password=password  elapsed=1.059s
     verify: Login successful.
```

Cracked: Magdy / password in **~1.1 s** against the short common-password list.

Q2 — Another Weak-Password User from `Q2/users`

```
[Q2] user=Mostafa  password=1234567890  elapsed=6.585s
     verify: Login successful.
```

Cracked: Mostafa / 1234567890 in **~6.6 s** (Magdy excluded).

Q3 — Dictionary Attack with `LeakedPWs100k`

Attack: brute-force the 100,000-word leaked list against the remaining users (Magdy and Mostafa already exposed). `LeakedPWs100k` carries no usernames, so there is no shortcut – each candidate is tried against every remaining user.

```
[Q3] 99946 pws across 8 workers (9 chunks)
[Q3] user=Akram  password=woopwoop  elapsed=8081.748s
     verify: Login successful.
```

Cracked: Akram / woopwoop in **~8082 s (~2 h 15 m)**. This is the slowest attack – each of the 100,000 candidates is hashed through the full 90,000-iteration scheme against every remaining user, so the result was produced by the parallel `bonus_q3_parallel.py` run (see Bonus).

Q4 — Password Reuse from `Q4/PwnedPWfile`

`Q4/PwnedPWfile` is a breach dump; every line is `breachedUser,password` (e.g. `Dyane,h4xY6hqZ`). A reuse attack therefore only needs to test each breached account's own password against the same account on our system. Six breach accounts overlap our live users, so the attack is six `login_hash` calls.

```
[Q4] 6 breach accounts overlap live users: ['Fathy', 'Islam', 'Kamal', 'May', 'Roaa', 'Waleed']
[Q4] user=Kamal  password=RizPbXsc  elapsed=0.080s
     verify: Login successful.
```

Cracked: Kamal / RizPbXsc in **0.08 s**.

> **Note on the earlier 2-hour run.** A previous version treated each whole line (including the `breachedUser,` prefix) as a password candidate, so no candidate could ever match. That forced the worst case – 5001 lines × 17 users × 90,000 SHA-256 iterations ≈ 7.6 billion hashes → NOT FOUND after 7570 s. Parsing the line into `user,password` and testing reuse per account reduces it to 0.08 s.

Q5 — Offline Attack on Unsalted Dump `Q5/HashedPWs`

Dump lines are `user,sha256(password)` where `password = leakedWord + 2 digits`. We hash every `leakedWord + dd` (`dd = 00..99`), match against the six users that appear in both the dump and our `users` file, then verify each hit against the login program – a dump collision only proves reuse on the breached site.

```
[Q5] overlap users: ['Fathy', 'Akram', 'Andrew', 'Ranya', 'Meina', 'Islam']
[Q5] 99946 dictionary words x 100 suffixes = 9994600 candidates
[Q5] leaked-hash scan done in 26.124s; 6 collisions
     candidate Islam/jason1276 -> Login failed: incorrect password.
     candidate Ranya/beograd43 -> Login failed: incorrect password.
     candidate Andrew/1011199065 -> Login successful.
[Q5] user=Andrew  password=1011199065  elapsed=26.543s
```

Cracked: Andrew / 1011199065 in **~27 s**. Islam and Ranya appear in the breach dump but did **not** reuse those passwords on our system, so they are correctly rejected and skipped.

Q6 — Offline Attack on Salted Dump `Q6/SaltedPWs`

Dump lines are `user,salt,sha256(salt + password)` where `password = leakedWord + 1 digit`. The per-user salt blocks any cross-user reverse index, so each overlapping user is attacked with their own salt; hits are then verified against the login program.

```
[Q6] overlap users: ['Peter', 'Akram', 'Islam', 'Salem']
[Q6] 99946 words x 10 suffixes per user
[Q6] salted-hash scan done in 5.789s; 4 collisions
     candidate Peter/clearwater9 -> Login successful.
[Q6] user=Peter  password=clearwater9  elapsed=5.922s
```

Cracked: Peter / clearwater9 in **~6 s**.

Bonus — Multi-threading the Slowest Program

The slowest attack is Q3 (full 100,000-word brute-force against the live login scheme). `bonus_q3_parallel.py` splits the dictionary across all CPU cores with `multiprocessing.Pool` (processes, not threads – SHA-256 is CPU-bound and the GIL blocks thread-level parallelism) and terminates on the first hit.

```
[Bonus] 99946 pws across 8 workers (9 chunks)
[Bonus] user=Akram  password=woopwoop  elapsed=8081.748s
        verify: Login successful.
```

Result: Akram / woopwoop in **~8082 s (~2 h 15 m)** across 8 worker processes. This is the parallel version of the Q3 attack and produced the Q3 result above.

Summary

Q	User	Password	Time	Verified
1	Magdy	`password`	1.06 s	✓
2	Mostafa	`1234567890`	6.59 s	✓
3	Akram	`woopwoop`	8081.75 s	✓
4	Kamal	`RizPbXsc`	0.08 s	✓
5	Andrew	`1011199065`	26.5 s	✓
6	Peter	`clearwater9`	5.92 s	✓
Bonus	Akram	`woopwoop`	8081.75 s	✓

How to Reproduce

```
# from Lab1\, using the project venv
$py = "..\.venv\Scripts\python.exe"
& $py q1_crack_magdy.py
& $py q2_crack_users.py
& $py q3_dictionary.py
& $py q4_pwned.py
& $py q5_hashed_dump.py
& $py q6_salted_dump.py
& $py bonus_q3_parallel.py
```